

Step 5 – Payout: `payoutClaim` (EIP-20)

The underwriter triggers the actual transfer of funds. This is the step where "\$221" stops being a number in storage and becomes a real balance change in Alice's wallet.

The actual sequence

1. Underwriter Safe calls `ClaimRegistry.payoutClaim`

- **Contract:** ClaimRegistry at `0x33333333333333333333333333333333CCCC`
- **Function:** `payoutClaim(uint256 claimId)`
- **Input:** `claimId = 1`
- **Caller:** `0x7777...aaaa` (the Underwriter Safe) — checked via `onlyRole(UNDERWRITER_ROLE)`
- **Imported contracts:** `IERC20`, `SafeERC20` (for the actual transfer, step 2 below)
- **Traversal, in order:**
 1. Checks `claims[1]` exists and `status == Approved` — true, per Step 4.
 2. Calls the internal `_enforceRateLimit(221 ether)`:
 - Checks whether the current rolling window has expired (`block.timestamp >= _windowStart + rateLimitWindowSeconds`); if so, resets `_windowStart = block.timestamp` and `_windowTotalPaid = 0`.
 - Checks `_windowTotalPaid + 221 ether <= maxPayoutPerWindow` — passes (this is the first payout in the window).
 - Updates `_windowTotalPaid += 221 ether`.
 3. Sets `claims[1].status = Paid`, `claims[1].processedAt = 1_800_600_000` — **this happens before the external token transfer below**, the standard checks-effects-interactions pattern, on top of the `nonReentrant` guard already in place.
 4. Calls `payoutToken.safeTransfer(0x1111...aaaa, 221 ether)` — an external call into `StableCoin` (step 2 below), since `payoutToken` is configured to the same `StableCoin` address used for premiums in this deployment.
 5. Emits `ClaimPayout(1, 0x1111...aaaa, 221 ether, 0x5555...eeee)`.

2. StableCoin.transfer — the actual dUSD movement

- **Contract:** StableCoin at 0x55555555555555555555555555555555EEEE
- **Function:** transfer(address to, uint256 amount) — plain inherited ERC20, not custom
- **Input:** to = 0x1111...aaaa, amount = 221 ether
- **Traversal:** checks _balances[0x3333...cccc] >= 221 ether (ClaimRegistry 's own balance — see the note below on where this came from), decrements it, increments _balances[0x1111...aaaa] by 221 ether.
- **Output:** bool (true); emits the EIP-20 event Transfer(0x3333...cccc, 0x1111...aaaa, 221 ether).

Important, not glossed over: this is a plain balance transfer, **not** a privileged mint. `StableCoin.mint / burn` (the `onlyVault`-gated functions) are never called anywhere in this trace. `ClaimRegistry` can only pay out dUSD it already holds — it has no minting rights at all. For `ClaimRegistry` to have had 221+ dUSD on hand *before* this transaction, someone (the insurer) must have already funded it, via their own `Vault` CDP position — see Step 6 for that separate, prerequisite flow.

Value trace summary – the complete before/after

```
BEFORE this step:
  StableCoin._balances[0x3333...cccc (ClaimRegistry)]:  >= 221 ether
  StableCoin._balances[0x1111...aaaa (Alice)]:          0 ether (in this trace)
```

```
claims[1].status:                Approved

AFTER this step:
  StableCoin._balances[0x3333...cccc]: decreased by 221 ether
  StableCoin._balances[0x1111...aaaa]: increased by 221 ether <- Alice now holds 221
dUSD
  claims[1].status:                Paid (terminal – cannot be paid again)
  _windowTotalPaid:                increased by 221 ether
```

This is the end of the "happy path" — Alice paid \$33, filed a claim, and received \$221. Everything from here (Step 6) covers what's deliberately *not* part of this trace: the CDP module, governance, and the gas paymaster.